

WealthCraftAgent 股票分析 Agent 架构设计文档

2352048 夏浩博 2351269 黄一和

2026 年 6 月 24 日

1 项目概述

WealthCraftAgent 是一个基于 .NET 10、WPF 与 Microsoft Semantic Kernel 的股票分析 Agent。项目面向 A 股分析场景，输入股票代码或股票名称后，系统从主营业务、K 线结构、财务报表、新闻动态四个角度生成分析，并支持缠论 RAG、历史相似走势 RAG 与会话内多轮追问。

本项目覆盖课程要求的五个核心要素：通过 OpenAI 兼容接口集成 LLM；通过 InvestAgentLoop 和会话编排实现 Thought-Action-Observation 推理循环；通过 Semantic Kernel 注册多个自定义工具；通过 ConversationMemory 与会话状态保存上下文；通过 WPF 桌面端提供用户交互界面。

2 总体架构

系统采用“桌面 UI + 核心 Agent 编排 + 工具/数据服务”的分层结构。WPF 负责输入、图表、历史记录与流式展示；Core 层负责 Agent 会话、子 Agent 调度、Semantic Kernel 调用、数据抓取和本地知识检索。

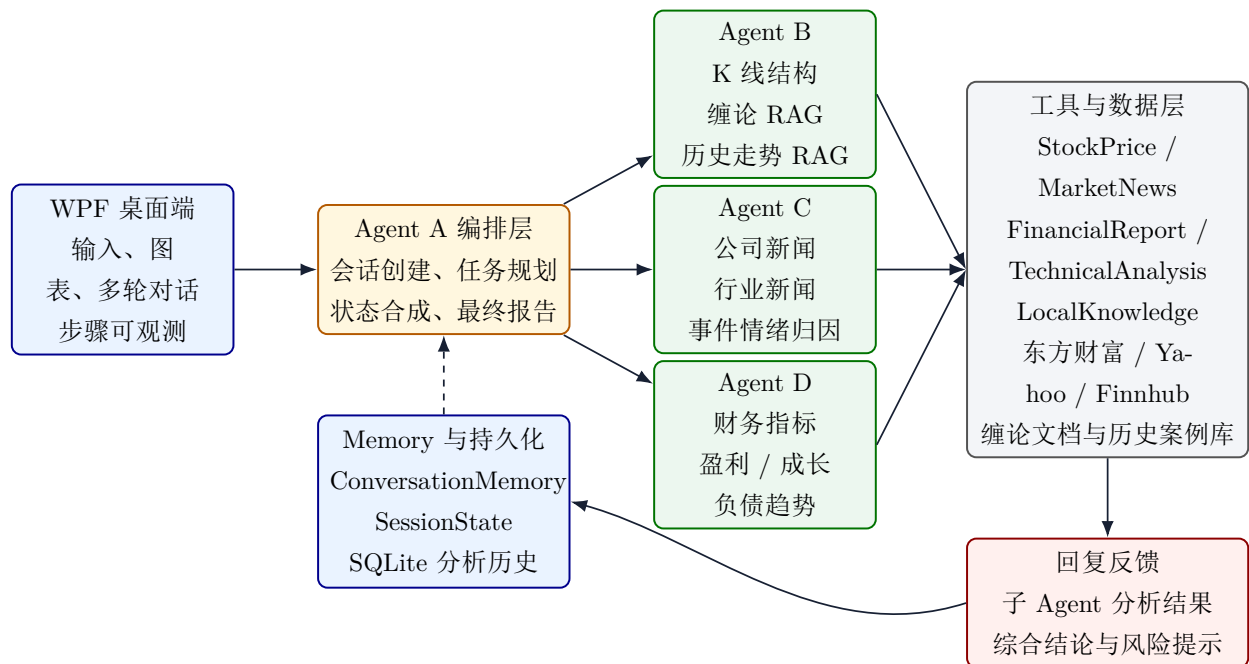


图 1: WealthCraftAgent 多 Agent 股票分析架构图

3 核心组件设计

3.1 LLM 集成

项目使用 Microsoft Semantic Kernel 统一管理 LLM 调用。依赖注入在 `AddInvestAgent` 中完成：读取 `AgentOptions` 中的模型、接口地址、API Key 和代理配置，创建 Kernel，并通过 `AddOpenAIChatCompletion` 接入 OpenAI 兼容接口。

LLM 调用分为两类：一类是 `InvestAgentLoop` 中的自动工具选择，用自动函数选择让模型决定调用哪个 Kernel Function；另一类是多 Agent 编排中的 `AgentPromptRunner`，负责流式输出、重试、推理标签过滤和图像输入降级。架构图中不把 LLM 单独画成业务节点，而是把它视为工具调用与回复生成过程中的基础能力：B/C/D 先连接到工具与数据层，工具层产生可用于回复的观察结果，再写回 Memory 与持久化模块供 Agent A 继续编排。

3.2 Agent Loop 与会话编排

项目同时保留了通用 ReAct 循环和更适合业务演示的确定性多 Agent 工作流。

- `InvestAgentLoop` 实现课程要求的 Thought–Action–Observation 循环：每轮先产生 Thought 步骤，调用 LLM，若返回工具调用则执行 Action 并记录 Observation，若无工具调用则输出最终 Response。
- `SessionAnalysisOrchestrator` 作为 Agent A，负责股票会话创建、初始分析并行调度、追问意图识别、子 Agent 派单和最终报告合成。
- Agent B/C/D 分别处理 K 线、新闻、财务，职责单一，便于扩展和答解释。

4 工具与数据设计

插件 / 服务	主要能力	设计说明
StockPrice	当前价格、历史日 K、月 K、股票搜索、资金流	为 Agent B 和 UI 图表提供行情基础数据；组合数据源优先适配 A 股。
FinancialReport	关键财务指标、利润分析	为 Agent D 提供 ROE、ROA、毛利率、净利率、增长率、负债率等序列。
MarketNews	新闻抓取、市场情绪	为 Agent C 提供公司新闻、行业新闻和情绪线索。
Technical Analysis	MA、RSI、MACD、交易信号	支持通用技术指标计算，也可被 ReAct 循环自动调用。
LocalKnowledge	缠论文档检索、缠论图片检索	从 docs/ 读取本地知识库，为缠论 RAG 提供文本片段和图例资源。

数据服务层使用组合模式：`CompositeStockDataService` 汇聚东方财富、Yahoo、Alpha Vantage、Finnhub 等来源，并配合 `FileHttpCache` 降低重复请求。历史相似走势 RAG 由

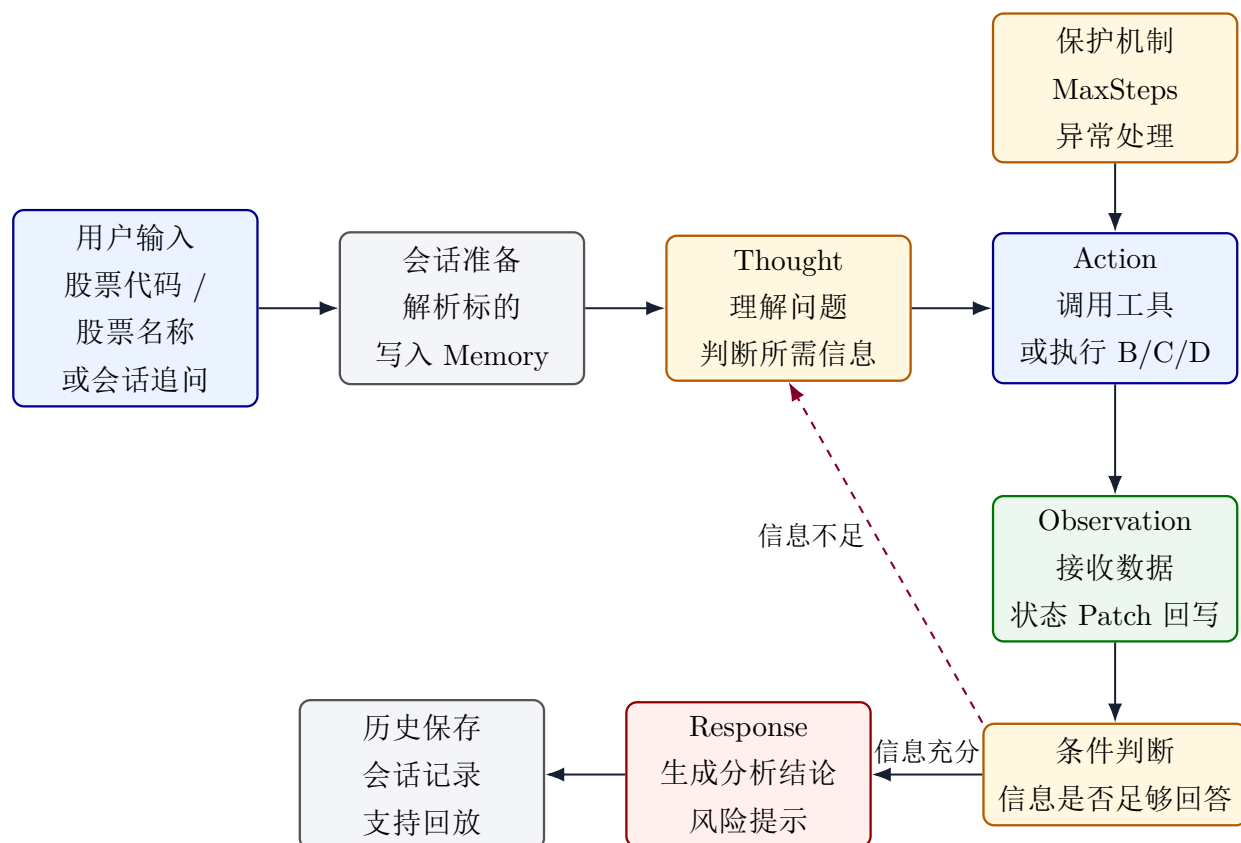


图 2: Agent 推理流程图

`HistoricalPatternService` 基于本地案例库提取特征并匹配类似结构，输出样本数量、相似原因和后续收益分布，避免把历史案例误写成确定性预测。

5 记忆、状态与可观测性

`ConversationMemory` 保存系统提示、用户消息、助手回复和工具返回，超过最大轮次时自动保留最近上下文，避免提示词无限膨胀。会话内结构化状态由 `AnalysisSessionState` 维护，包括股票信息、K 线缓存、新闻列表、财务序列、各子 Agent 结果、最终报告等。

可观测性通过 `AgentStep` 实现。每个 Thought、Action、Observation、Response 都会通过 `IAnalysisStreamingObserver` 推送到 WPF，用户能够看到 Agent 正在抓取数据、检索知识库、生成分析或处理异常。

6 异步、错误处理与扩展性

项目大量使用 `async/await`：股票数据、新闻、财务指标、LLM 流式输出、初始 B/C/D 并行分析均为异步流程。初始分析中，Agent A 对 B/C/D 使用 `Task.WhenAll` 并发执行，提高首轮报告生成速度。

错误处理分为三层：LLM 调用失败时返回用户可读错误；工具或子 Agent 失败时记录日志并跳过该模块继续分析；数据不足时在提示词和最终报告中显式说明不确定性。扩展新工具时，只需实现新的服务或 Kernel 插件，并在 DI 中注册；扩展新子 Agent 时，实现 `ISubAgentService` 即可

纳入编排。

7 课程要求对应关系

课程要求	项目实施
LLM 集成	Semantic Kernel + OpenAI ChatCompletion，支持流式输出和自动函数选择。
推理循环	InvestAgentLoop 实现 ReAct; SessionAnalysisOrchestrator 实现业务化多 Agent workflows。
工具调用	注册 5 个插件，多于课程要求的 3 个自定义工具。
Memory	ConversationMemory、AnalysisSessionState 与 SQLite 历史记录共同维护上下文。
用户界面	WPF 桌面端，包含 K 线图、新闻、财务、聊天和历史回放。
加分项	多 Agent 协作、RAG、流式输出、可观测步骤、单元测试。